CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

Angelica Dias Folgado

RU: 4937552

RAÍZES DO NORDESTE

Projeto de API REST para pedidos multicanal, estoque por unidade e pagamento simulado

Projeto Multidisciplinar - Trilha Back-End

2026

Apresentação e identificação

Relatório acadêmico de Angelica Dias Folgado, RU 4937552. O projeto implementa uma API REST para a rede fictícia Raízes do Nordeste, com foco no fluxo de pedido, pagamento mock e acompanhamento operacional.

esta versão reúne o código e as evidências locais disponíveis em 11/10/2026. O roteiro fornecido com o projeto foi adotado como referência oficial desta entrega por orientação do aluno. Curso, polo e cidade não foram informados e não foram presumidos. A instalação em cópia limpa foi validada. O conteúdo final está consolidado em um único commit de Angelica Dias Folgado, pronto para substituir a história anterior no repositório público.

O material distingue funcionalidades implementadas de propostas futuras. Os resultados de desempenho se limitam ao ambiente local e não caracterizam garantia de produção.

Sumário

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER 1

Projeto de API REST para pedidos multicanal, estoque por unidade e pagamento simulado 1

Apresentação e identificação 2

1. Introdução, contexto e objetivos 4

2. Requisitos e priorização 5

3. Propostas fora do MVP 10

4. Arquitetura e decisões técnicas 12

5. Modelagem do sistema 13

6. Funcionalidade crítica 19

7. Contratos da API 21

8. Segurança, privacidade e auditoria 29

9. Plano de testes 33

10. Resultados e desempenho 39

11. Instalação e recuperação 41

12. Rastreabilidade 43

13. Conclusão e situação da entrega 46

14. Referências e artefatos consultados 47

1. Introdução, contexto e objetivos

Uma rede de alimentação com matriz e unidades precisa integrar pedidos recebidos por diferentes canais, disponibilidade local de produtos e acompanhamento do preparo. Registros inconsistentes podem gerar venda sem saldo e alterações indevidas de status.

O objetivo é demonstrar um backend com autenticação, autorização por perfil, persistência real, reserva transacional e integração simulada de pagamento. Os canais APP, WEB, TOTEM, BALCAO e PICKUP são registrados no domínio e podem ser usados para filtrar pedidos.

O MVP escolhido é o Fluxo A: Pedido → Pagamento mock → Status. Essa escolha permite validar uma operação completa sem depender de adquirentes, dados de cartão ou serviços financeiros reais. TypeScript, Express, Prisma e SQLite viabilizam uma demonstração local reproduzível.

ADMIN representa a administração da rede. Não há entidade Matriz nem vínculo obrigatório de funcionários a uma unidade; os perfis operacionais consultam a rede. Restrição operacional por unidade é uma evolução futura.

2. Requisitos e priorização

P1 identifica requisitos essenciais; P2, qualidade e consistência; P3, evolução conceitual. Os critérios abaixo especificam o comportamento esperado. O estado final e suas evidências são detalhados na seção de rastreabilidade.

4. Requisitos funcionais

ID	Requisito	Prioridade / estado	Critério de aceite ou especificação da proposta
RF01	Autenticar por e-mail e senha e emitir JWT com validade definida.	P1 / Implementado	Credenciais válidas retornam token; inválidas retornam 401; dados ausentes retornam erro de validação. Senha e hash não aparecem no retorno.
RF02	Autorizar operações por perfil e proprietário do pedido.	P1 / Implementado	Ausência de token retorna 401; perfil indevido retorna 403; cliente não consulta nem paga pedido de outro cliente.
RF03	Cadastrar cliente com e-mail único e senha protegida.	P3 / Proposto	Descrever validação, prevenção de duplicidade e atribuição exclusiva de CLIENTE no cadastro público. O seed continua sendo a origem das contas de demonstração.
RF04	Consultar unidades da rede.	P1 / Implementado	Retornar identificação, nome e cidade das unidades cadastradas. Paginação implementada pelo RF16.
RF05	Consultar cardápio e disponibilidade por unidade.	P1 / Implementado	Filtro por unidade retorna apenas seu cardápio/estoque; unidade inexistente retorna 404; saldo de outra unidade não é agregado.
RF06	Criar pedido com itens e valores calculados no servidor.	P1 / Implementado	Pedido válido retorna 201; itens repetidos são agrupados; IDs/quantidades inválidos são rejeitados; preço enviado pelo consumidor não controla o total.
RF07	Registrar e filtrar o canal de origem do pedido.	P1 / Implementado	Exigir APP, TOTEM, BALCAO, PICKUP ou WEB; ausência/valor inválido retorna 400/422; filtro retorna somente pedidos do canal solicitado.
RF08	Reservar estoque da unidade transacionalmente.	P1 / Implementado	Falta de saldo retorna 409; falha em qualquer item reverte toda a reserva; pedidos concorrentes não tornam o saldo negativo.
RF09	Consultar pedidos com paginação e isolamento por cliente.	P1 / Implementado	Validar page/limit, limitar a 100 itens por página e retornar metadados; cliente vê apenas seus pedidos; perfis operacionais consultam a rede.
RF10	Solicitar e registrar pagamento mock por contrato de gateway.	P1 / Implementado	Preservar aprovação/recusa atuais; mover simulação para adaptador da infraestrutura, acessado por interface da aplicação; comprovar envio, retorno e correlação com o pedido.
RF11	Tratar recusa, falha técnica e repetição de pagamento.	P1 / Implementado	Recusa mantém pedido aguardando pagamento; nova tentativa é permitida; pedido aprovado não é cobrado novamente. Falha técnica simulada retorna erro padronizado e não aprova pedido nem grava pagamento aprovado.
RF12	Atualizar status respeitando a sequência operacional.	P1 / Implementado	Permitir EM_PREPARO → PRONTO → ENTREGUE apenas a COZINHA/ADMIN; rejeitar saltos/regressões com 409.

ID	Requisito	Prioridade / estado	Critério de aceite ou especificação da proposta
RF13	Cancelar pedido ainda não pago e devolver estoque.	P1 / Implementado	COZINHA/ADMIN pode mudar AGUARDANDO_PAGAMENTO para CANCELADO; restituição ocorre uma vez; pedido cancelado não aceita pagamento.
RF14	Auditar ações sensíveis e acessos operacionais a pedidos.	P1 / Implementado	Manter logs de login, criação, pagamento e status; acrescentar registro de consultas operacionais com ator, ação e instante, sem senha, JWT ou cópia integral de dados pessoais. Definir evidência reproduzível de auditoria para o avaliador.
RF15	Registrar e permitir revogar consentimento opcional.	P1 / Implementado	Usuário autenticado registra apenas sua própria decisão para finalidade identificada, com data e versão do aviso; revogação fica rastreável e não bloqueia pedidos comuns. Consentimento de demonstração inserido pelo seed deve ser identificado como fictício.
RF16	Paginar consultas de produtos e unidades.	P2 / Implementado	Aceitar page/limit, com padrão 1/10 e máximo 100; retornar dados e metadados, preservando filtros. Atualizar Swagger, Postman e consumidores de teste para o novo formato.
RF17	Administrar produtos e unidades.	P3 / Proposto	Especificar criação/edição/inativação por ADMIN; impedir remoções que quebrem pedidos históricos e manter identificação estável.
RF18	Registrar entradas e ajustes administrativos de estoque.	P3 / Proposto	Descrever movimentação com unidade, produto, quantidade, tipo, motivo, autor e data; bloquear saída sem saldo e preservar histórico. O MVP já implementa saída por pedido e devolução por cancelamento.
RF19	Acumular e resgatar pontos mediante adesão opcional.	P3 / Proposto	Especificar saldo, lançamentos, acúmulo único por pedido entregue e resgate sem saldo negativo; detalhamento na seção 7.
RF20	Aplicar promoções/campanhas elegíveis.	P3 / Proposto	Especificar vigência, unidade, canal, produtos elegíveis e desconto calculado pelo servidor; detalhamento na seção 7.

RF15 não implica implementar o programa de pontos: nesta entrega, o consentimento será uma capacidade técnica demonstrável para uma finalidade opcional claramente descrita. A política documental deverá manter coerência com as funcionalidades realmente executadas.

5. Requisitos não funcionais

ID	Requisito	Prioridade / estado	Critério de aceite
RNF01	Separar domínio, aplicação, infraestrutura e API.	P1 / Implementado	Domínio/aplicação não importam Express ou Prisma; integrações usam interfaces; controllers tratam entrada/saída e delegam regras.
RNF02	Persistir dados com integridade e migrações reproduzíveis.	P1 / Implementado	Banco recriado por migrações; PK/FK e unicidade coerentes; seed repetido não apaga pedidos nem repõe estoque consumido.
RNF03	Proteger credenciais e segredos.	P1 / Implementado	Senhas com bcrypt, JWT assinado e validado, segredo externo ao código e ignorado pelo versionamento; responses sem hashes/tokens de terceiros.
RNF04	Explicitar finalidade, minimização e ciclo de vida dos dados.	P1 / Implementado	Inventário por campo e finalidade, base de tratamento a fundamentar, controle de acesso, estratégia de retenção/anonimização e procedimento de revogação; distinguir política proposta de mecanismo implementado.
RNF05	Padronizar erros e validar entradas.	P1 / Implementado	Envelope com error, message, details, timestamp, path e requestId; sem stack trace no cliente; status coerentes para validação, acesso e conflito.
RNF06	Manter contratos OpenAPI completos e compatíveis.	P1 / Implementado	Cada operação tem parâmetros, permissões, exemplos de request quando aplicável, response de sucesso e erro, além dos códigos possíveis.
RNF07	Permitir instalação e execução local reproduzíveis.	P1 / Implementado	Em diretório limpo, instruções do README instalam dependências, configuram ambiente, migram, populam e iniciam a API. Revalidar ao fechar a entrega.
RNF08	Garantir evidências de teste reproduzíveis.	P1 / Implementado	Build e testes passam; coleção organizada em pastas com pelo menos 10 cenários, 6 positivos e 4 negativos, cobrindo os casos obrigatórios e evidência de auditoria; cada cenário possui pré-condição e resultado esperado.
RNF09	Limitar custo das consultas e avaliar concorrência.	P2 / Verificado localmente	Aplicar RF16; preservar testes de última unidade e pagamento concorrente. Proposta de medição local: 20 clientes concorrentes durante 60 s em consultas, registrar ambiente, p95 e erros; meta inicial p95 ≤ 1 s e menos de 1% de 5xx. Meta atingida no ensaio local posterior; não é garantia de produção.
RNF10	Recuperar a operação após falhas locais.	P2 / Verificado localmente	Após reiniciar a API, pedidos persistidos permanecem consultáveis; manter health de processo e encerramento controlado; documentar backup/restauração do SQLite e verificar restauração em cópia de teste. Sem compromisso de disponibilidade 24×7.

ID	Requisito	Prioridade / estado	Critério de aceite
RNF11	Tolerar falhas do gateway simulado.	P1 / Implementado	Simular indisponibilidade e timeout determinísticos, sem rede financeira real; estado do pedido e reserva permanecem coerentes; registrar resultado técnico sem transformar falha em aprovação.
RNF12	Preservar rastreabilidade entre requisito, código e evidência.	P1 / Implementado	Cada requisito implementado aponta para caso de uso/rota e teste; cada proposta é identificada como tal; diagramas e texto não apresentam funcionalidade inexistente como executável.

3. Propostas fora do MVP

7. Funcionalidades propostas: especificação conceitual

7.1 Cadastro e gestão administrativa

Cadastro de cliente receberia nome, e-mail e senha, normalizaria o e-mail, impediria duplicidade e nunca aceitaria ADMIN como papel escolhido pelo consumidor. A adesão opcional teria campo separado. Perfis operacionais seriam criados por administrador autorizado. Produtos/unidades seriam inativados em vez de apagados quando tivessem vínculo histórico.

Rotas candidatas, **não implementadas**: POST /api/usuarios, POST /api/unidades, PATCH /api/unidades/{id}, POST /api/produtos e PATCH /api/produtos/{id}. Funcionários por unidade exigiriam vínculo de autorização adicional no modelo.

7.2 Movimentação administrativa de estoque

Modelo proposto: MovimentacaoEstoque com produtoId, unidadeId, tipo (ENTRADA/SAIDA/AJUSTE), quantidade, motivo, usuarioId e criadoEm. Saldo e lançamento seriam atualizados na mesma transação, sem saldo negativo. Ajuste usaria variação assinada e justificativa obrigatória, mantendo o saldo anterior rastreável.

Rotas candidatas, **não implementadas**: POST /api/estoques/movimentacoes e GET /api/estoques/movimentacoes?unidadeId=.... O extrato seria paginado e restrito a administradores autorizados.

7.3 Fidelização

Política de demonstração proposta, não exigência literal do roteiro: conceder 1 ponto por R$ 1,00 efetivamente pago em pedido entregue, descartando a fração. Para R$ 38,50, conceder 38 pontos. Só acumular após adesão opcional ativa; não conceder pontos por pedidos recusados ou cancelados.

Resgate proposto: blocos de 100 pontos concedem R$ 5,00 de desconto, limitados ao saldo disponível e a 20% do subtotal elegível. Se o subtotal for R$ 20,00, um bloco de R$ 5,00 não cabe no limite e não será aplicado. Débito e reserva dos pontos devem ocorrer transacionalmente; cancelamento anterior ao pagamento devolve os pontos reservados uma única vez.

Modelo proposto: ContaFidelidade e LancamentoPontos (crédito, débito, devolução), com pedido de origem e proteção contra crédito duplicado. O acúmulo incide sobre o valor efetivamente pago após desconto. Revogar adesão impede novos créditos e resgates; tratamento do saldo e retenção deve ser explicitado na política proposta, sem exclusão indiscriminada do histórico de pedidos.

Rotas candidatas, **não implementadas**: GET /api/fidelidade/saldo, GET /api/fidelidade/extrato e solicitação de resgate durante criação de pedido. O próprio cliente acessaria seu saldo/extrato; o resgate seria recalculado pelo servidor.

7.4 Promoções e campanhas

Modelo proposto: Campanha com nome, início/fim, situação, unidades/canais elegíveis, produtos aplicáveis e desconto fixo ou percentual. O servidor validaria a vigência e calcularia o desconto em centavos, sem total negativo, gravando a campanha e o desconto efetivamente aplicados no pedido.

Para reduzir ambiguidades na primeira versão proposta, permitir apenas uma campanha por pedido e não acumulá-la com resgate de pontos. Registrar o desconto no pedido evita que edição futura da campanha altere pedidos anteriores. Datas seriam armazenadas em UTC e apresentadas no fuso da operação.

Rotas candidatas, **não implementadas**: gestão de /api/campanhas por ADMIN e campo de campanha na criação de pedido. Campanha inexistente, expirada ou inelegível produziria erro de negócio documentado.

4. Arquitetura e decisões técnicas

Arquitetura e decisões

Camada	Responsabilidade e ponto de entrada
Domínio	Entidades, enums e erros; contratos independentes de Express e Prisma
Aplicação	Casos de uso; validação, regras, paginação e portas Store, Security, PaymentGateway
Infraestrutura	Adaptadores; Prisma/SQLite, bcrypt, JWT, auditoria e gateway mock
API	Rotas, controllers e middlewares; HTTP, autenticação, perfis, serialização e OpenAPI
Composição	composition.ts injeta implementações nas portas
Inicialização	server.ts valida configuração, conecta banco, inicia listener e trata encerramento

A chamada percorre rota → middleware → controller → caso de uso → porta → adaptador. A aplicação conhece interfaces; a composição conhece implementações concretas. As entidades são interfaces TypeScript, sem métodos de negócio. Veja os quatro diagramas.

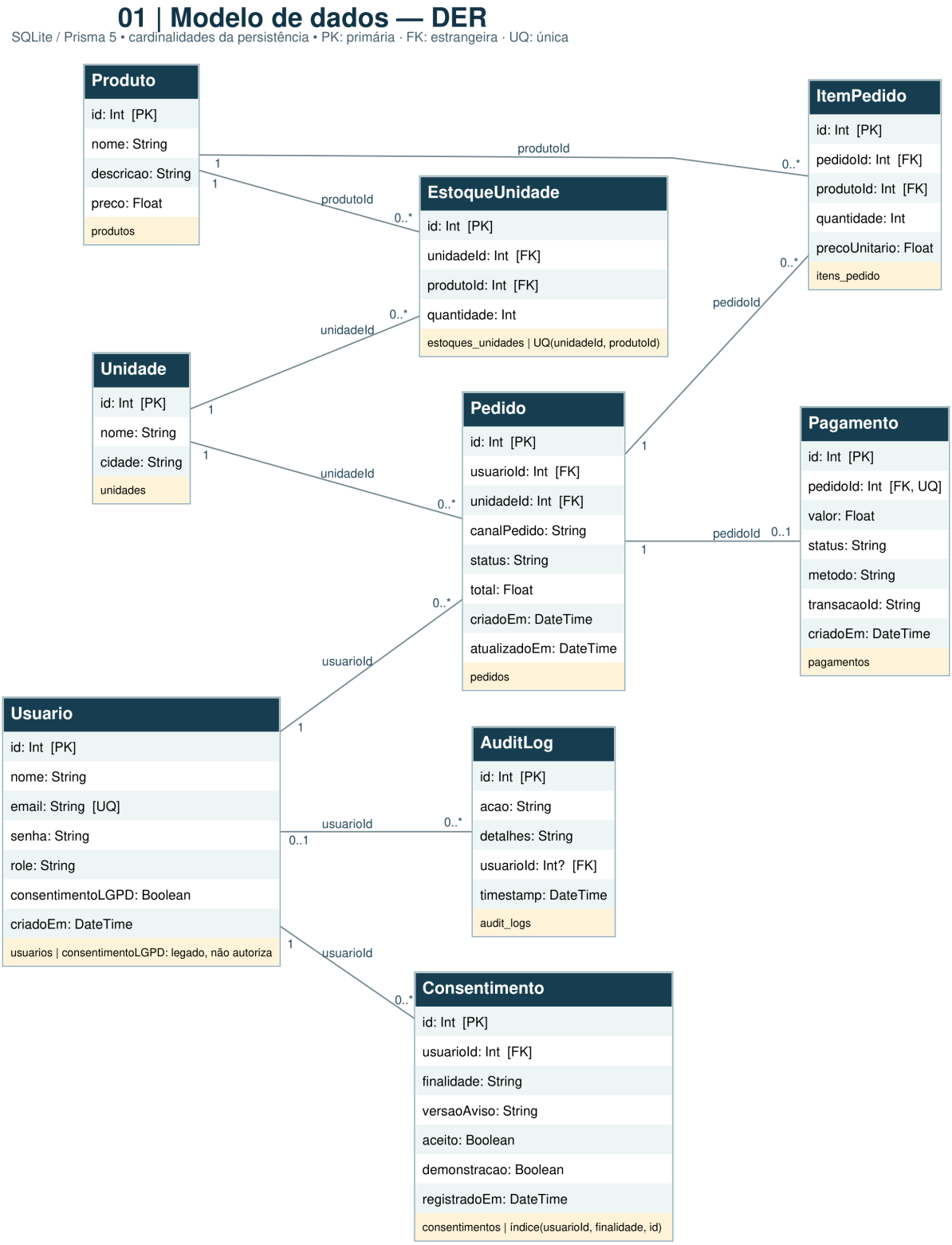
SQLite atende à execução local demonstrativa, com migrações versionadas. Estados são strings persistidas e validados pela aplicação. Preços usam Float na persistência; o cálculo do total usa centavos inteiros. O pagamento é local e determinístico: não há cobrança financeira, webhook, estorno ou credenciais de adquirente. O adaptador atual é chamado dentro da transação; uma integração de rede futura exigirá redesenhar duração de transações, idempotência e conciliação.

5. Modelagem do sistema

Os diagramas representam a implementação atual. Fontes: schema Prisma, interfaces de domínio, rotas e casos de uso do projeto. Elaboração a partir dos artefatos técnicos locais, 2026.

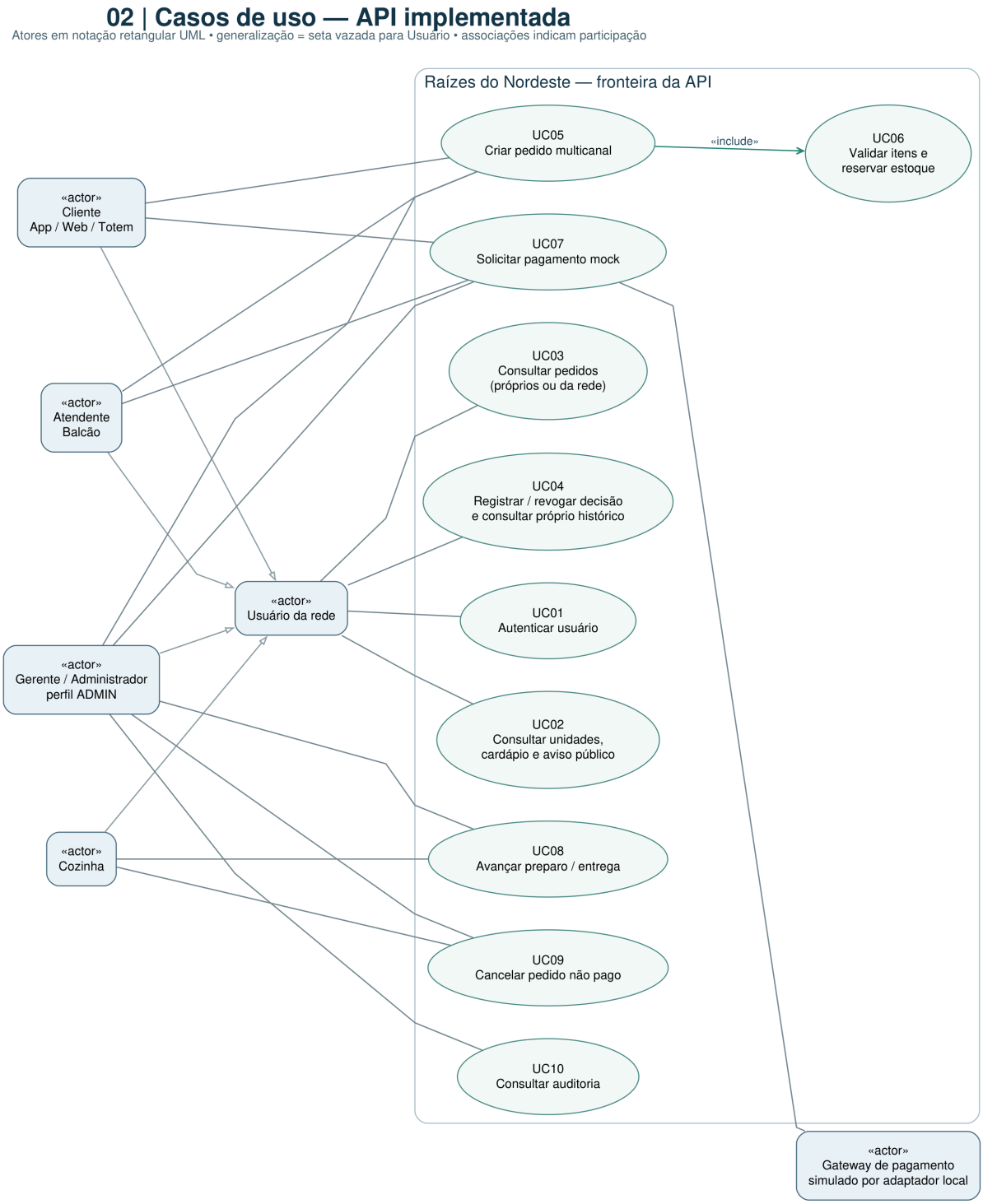
O DER utiliza cardinalidades de persistência; a aplicação exige pelo menos um item por pedido. As classes são interfaces TypeScript, com comportamento nos casos de uso. O gateway aparece como participante de apoio, mas é executado localmente como mock.

Figura 1 - Modelo de dados (DER)



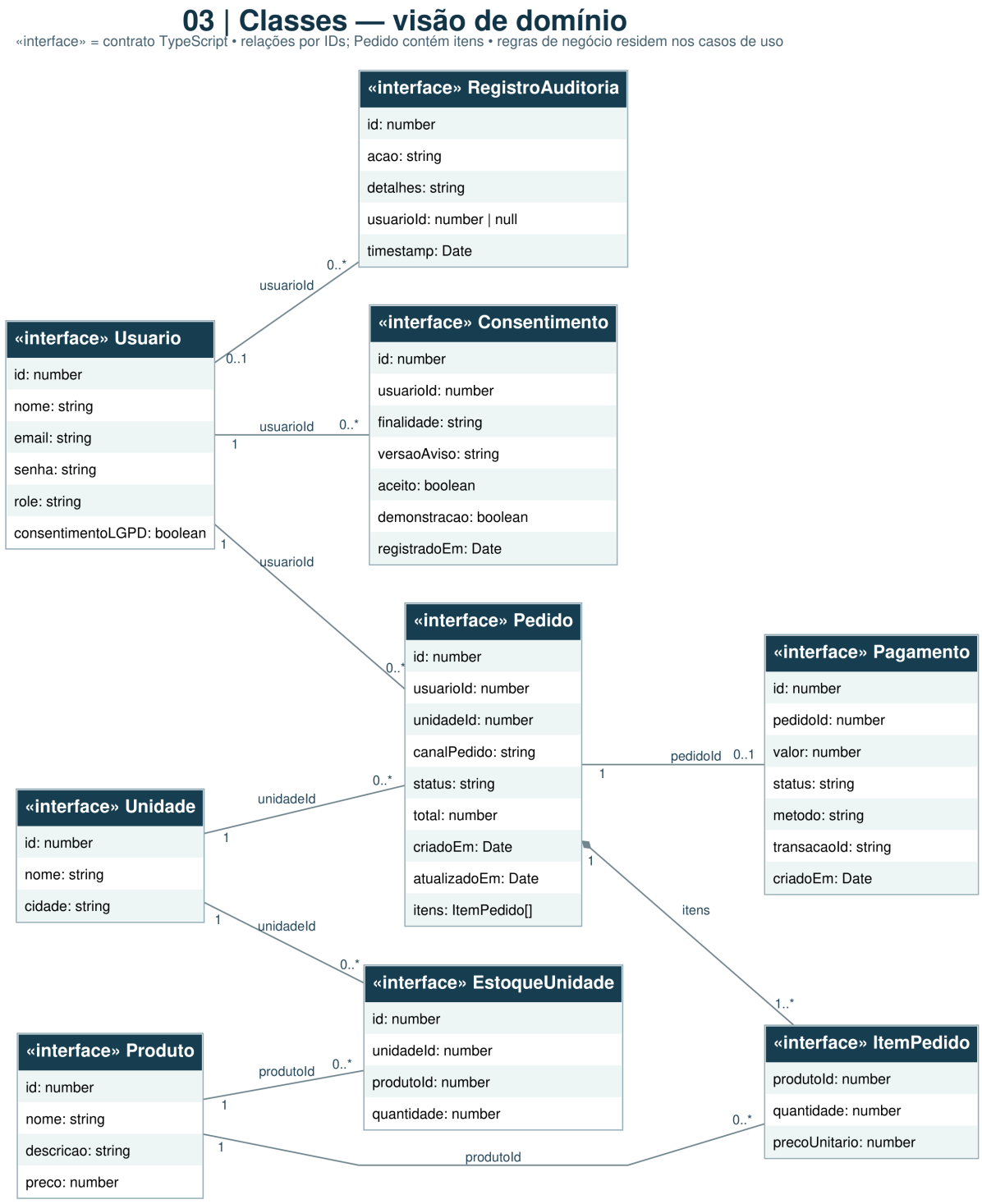
Fonte: artefatos do projeto, 2026.

Figura 2 - Casos de uso



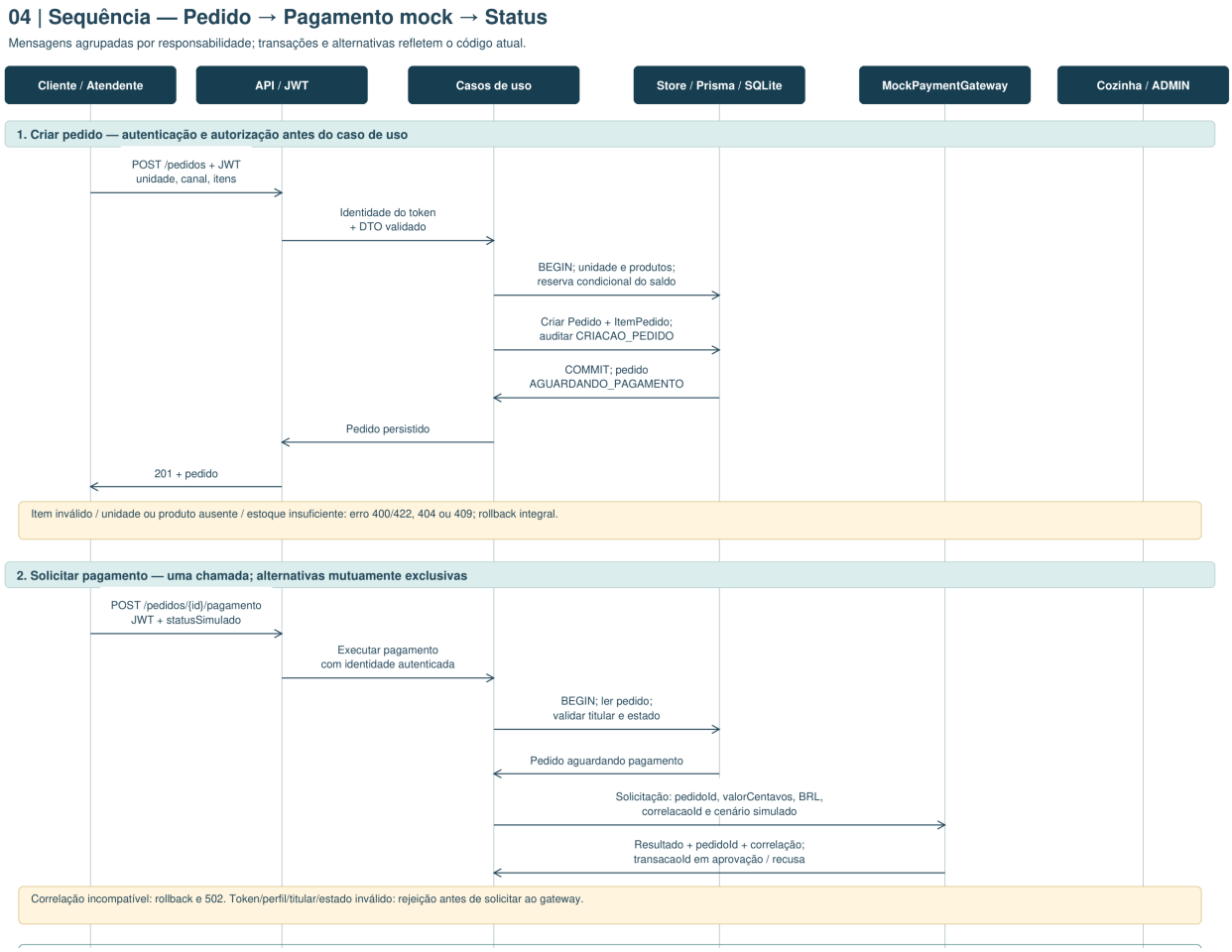
Fonte: artefatos do projeto, 2026.

Figura 3 - Classes de domínio



Fonte: artefatos do projeto, 2026.

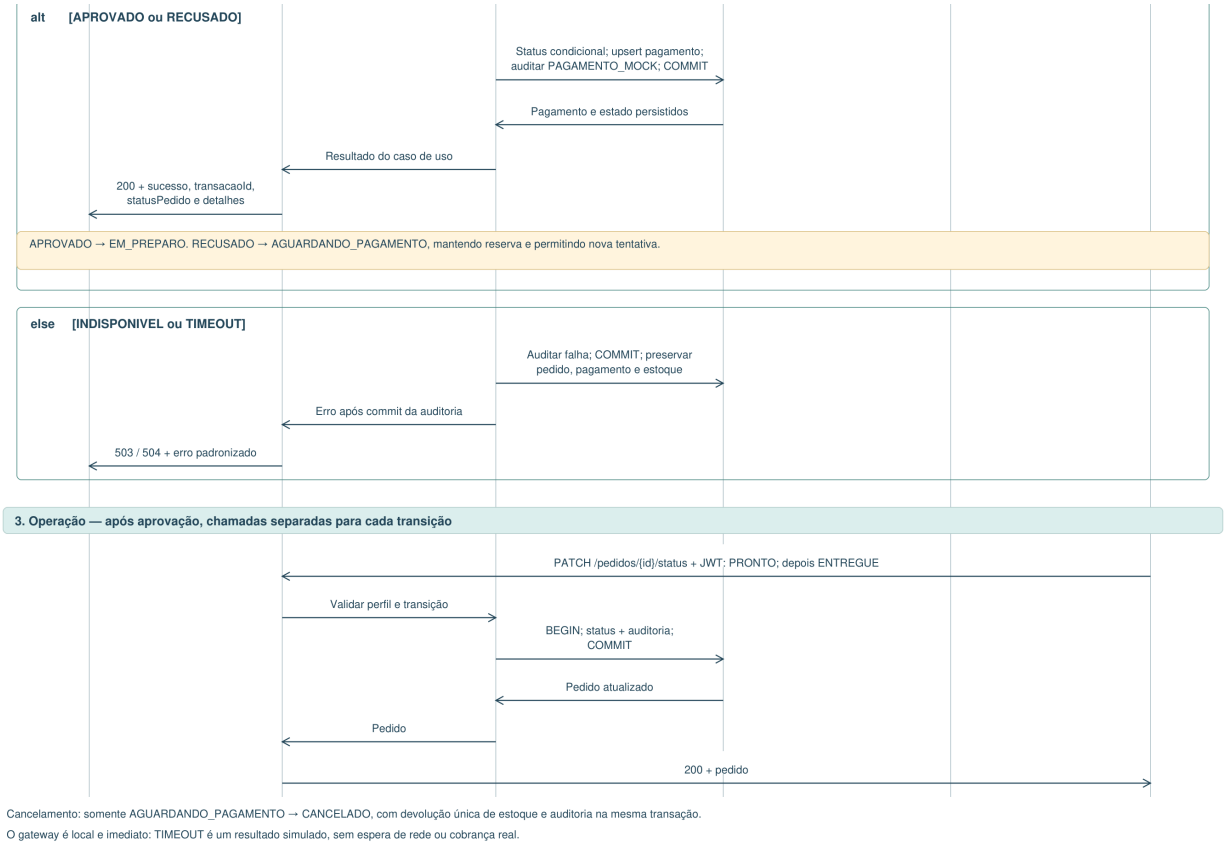
Figura 4 - Sequência do fluxo crítico (parte 1/2)



Fonte: artefatos do projeto, 2026. Partes consecutivas do mesmo diagrama.

Figura 4 - Sequência do fluxo crítico (parte 2/2)

Participantes: Cliente/Atendente, API/JWT, Casos de uso, Store/Prisma/SQLite, MockPaymentGateway e Cozinha/ADMIN.



Fonte: artefatos do projeto, 2026. Partes consecutivas do mesmo diagrama.

6. Funcionalidade crítica

Funcionalidade crítica: pedido e pagamento

Objetivo: registrar pedido multicanal, reservar saldo da unidade e processar o pagamento mock mantendo consistência entre pedido, estoque e auditoria.

Atores: CLIENTE (seus pedidos), ATENDENTE e ADMIN (criação e pagamento); COZINHA e ADMIN (transições operacionais). Participantes técnicos: API, casos de uso, Store/Prisma e gateway mock. O atendente cria com sua própria identidade do JWT; não escolhe outro titular no corpo.

Pré-condições: usuário do seed com JWT válido, perfil permitido, unidade/produtos cadastrados e saldo suficiente. Para pagamento, pedido existente em AGUARDANDO_PAGAMENTO; cliente deve ser seu proprietário. Consentimento de fidelidade não é pré-condição.

Fluxo principal:

1. Autenticar em POST /api/auth/login; copiar accessToken para Authorize no Swagger.

2. Consultar unidades e produtos para escolher IDs e verificar saldo.

3. Enviar POST /api/pedidos com unidade, canal e 1 a 100 itens. Identidade vem do token, preço vem do banco; produtos repetidos são agrupados.

4. Na mesma transação, validar referências, reservar saldo condicionalmente, calcular total, persistir pedido/itens e auditar. Retornar 201 e AGUARDANDO_PAGAMENTO.

5. Enviar POST /api/pedidos/{id}/pagamento com {"statusSimulado":"APROVADO"}. Validar identidade/estado e enviar pedido, centavos, moeda BRL e correlação ao gateway.

6. Validar correlação, atualizar estado condicionalmente, gravar pagamento e auditoria. Retornar 200 com sucesso: true e statusPedido: "EM_PREPARO".

7. Com token COZINHA/ADMIN, enviar PATCH /api/pedidos/{id}/status primeiro com {"status":"PRONTO"}, depois {"status":"ENTREGUE"}.

Exemplo do corpo de criação:

```
{"unidadeId":1,"canalPedido":"WEB","itens":[{"produtoId":1,"quantidade":1}]}
```

Pós-condições de sucesso: pedido e itens persistidos, estoque reservado somente na unidade escolhida, pagamento aprovado associado ao pedido e eventos auditados. O pagamento aprovado muda para preparo; a entrega depende de chamadas operacionais posteriores.

Alternativa/exceção	Resultado e preservação de dados
Entrada inválida / token ausente / perfil indevido	400 ou 422 / 401 / 403; operação rejeitada
Referência inexistente	404
Saldo insuficiente ou conflito de estado	409; reserva parcial da criação é revertida
RECUSADO	200, sucesso: false; pedido aguarda pagamento, reserva permanece; nova tentativa permitida
INDISPONIVEL / TIMEOUT	503 / 504; auditoria da falha confirmada, pedido/estoque/pagamento anterior preservados

Alternativa/exceção	Resultado e preservação de dados
Correlação incompatível	502 e rollback da transação
Tentativa após aprovação/cancelamento	409; não registra segunda aprovação
Cancelar antes do pagamento	COZINHA/ADMIN envia CANCELADO; devolução de estoque e auditoria transacionais, uma única vez
Falha inesperada	Envelope padronizado sem stack trace; não prometer sucesso ao consumidor

O pagamento guarda a última tentativa no registro único do pedido; a auditoria conserva eventos. Uma recusa não libera saldo. Não há expiração automática de reservas nem cancelamento de pedido pago. Para erros detalhados, exemplos e permissões de cada operação, use o contrato OpenAPI e a matriz T01-T32.

7. Contratos da API

Base local: http://localhost:3000/api. Swagger interativo: http://localhost:3000/api-docs. O contrato completo possui 12 operações e 81 exemplos explícitos. Os exemplos abaixo são fictícios, extraídos do contrato do projeto; tokens ilustrativos não são credenciais válidas.

POST /auth/login

Autenticar Utilizador

Autenticação: Pública.

Request (exemplo):

```
{
  "email": "maria@cliente.com",
  "senha": "123456"
}
```

Respostas documentadas: 200, 400, 401, 413, 422, 500

Response 200 (exemplo):

```
{
  "accessToken": "JWT_FICTICIO_OBTENHA_UM_TOKEN_PELO_LOGIN",
  "tokenType": "Bearer",
  "expiresIn": 28800,
  "usuario": {
    "id": 1,
    "nome": "Maria da Silva",
    "email": "maria@cliente.com",
    "role": "CLIENTE"
  }
}
```

GET /produtos

Consultar Ementa e Existências

Autenticação: Pública.

Parâmetro unidadeId (query): Filtrar cardápio e estoque por unidade.

Parâmetro page (query):

Parâmetro limit (query):

Respostas documentadas: 200, 404, 409, 422, 500

Response 200 (exemplo):

```
{
  "dados": [
    {
      "id": 1,
      "nome": "Baião de Dois Completo",
      "descricao": "Queijo coalho e carne de sol",
      "preco": 38.5,
      "estoques": [
        {
          "id": 1,
          "unidadeId": 1,
          "produtoId": 1,
          "quantidade": 25,
```

```
          "unidade": {
            "id": 1,
            "nome": "Unidade Recife Olinda",
            "cidade": "Recife"
          }
        }
      ]
    }
  ],
  "paginacao": {
    "pagina": 1,
    "limite": 10,
    "total": 1,
    "totalPaginas": 1
  }
}
```


POST /pedidos

Criar novo Pedido Multi-canal

CLIENTE, ATENDENTE ou ADMIN. Pedido pertence ao usuário autenticado. Itens repetidos são agrupados. Falhas revertem toda a reserva.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Request (exemplo):


```
{
  "unidadeId": 1,
  "canalPedido": "TOTEM",
  "itens": [
    {
      "produtoId": 1,
      "quantidade": 2
    },
    {
      "produtoId": 2,
      "quantidade": 1
    }
  ]
}
```


Respostas documentadas: 201, 400, 401, 403, 404, 409, 413, 422, 500

Response 201 (exemplo):


```
{
  "id": 1,
  "usuarioId": 1,
  "unidadeId": 1,
  "canalPedido": "TOTEM",
  "status": "AGUARDANDO_PAGAMENTO",
  "total": 91.0,
  "criadoEm": "2026-10-11T12:00:00.000Z",
  "atualizadoEm": "2026-10-11T12:00:00.000Z",
  "itens": [
    {
      "id": 1,
      "pedidoId": 1,
      "produtoId": 1,
      "quantidade": 2,
      "precoUnitario": 38.5
    },
    {
      "id": 2,
      "pedidoId": 1,
      "produtoId": 2,
      "quantidade": 1,
```

```
      "precoUnitario": 14.0
    }
  ]
}
```


GET /pedidos

Listar Pedidos com paginação e filtro por Canal

CLIENTE vê apenas seus pedidos. ATENDENTE, COZINHA e ADMIN consultam todos. Máximo de 100 itens por página. Consultas geram auditoria de ator, escopo e filtros, sem cópia dos pedidos no log.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Parâmetro canalPedido (query):

Parâmetro page (query):

Parâmetro limit (query):

Respostas documentadas: 200, 400, 401, 409, 422, 500

Response 200 (exemplo):


```
{
  "dados": [
    {
      "id": 1,
      "usuarioId": 1,
      "unidadeId": 1,
      "canalPedido": "TOTEM",
      "status": "AGUARDANDO_PAGAMENTO",
      "total": 91.0,
      "criadoEm": "2026-10-11T12:00:00.000Z",
      "atualizadoEm": "2026-10-11T12:00:00.000Z",
      "itens": [
        {
          "id": 1,
          "pedidoId": 1,
          "produtoId": 1,
          "quantidade": 2,
          "precoUnitario": 38.5,
          "produto": {
            "id": 1,
            "nome": "Baião de Dois Completo",
            "descricao": "Queijo coalho e carne de sol",
            "preco": 38.5
          }
        },
        {
          "id": 2,
          "pedidoId": 1,
          "produtoId": 2,
          "quantidade": 1,
          "precoUnitario": 14.0,
          "produto": {
            "id": 2,
            "nome": "Tapioca Tradicional de Manteiga",
            "descricao": "Tapioca artesanal nordestina",
            "preco": 14.0
          }
        }
      ],
      "pagamento": null
    }
  ],
  "paginacao": {
```

```
    "pagina": 1,
    "limite": 10,
    "total": 1,
    "totalPaginas": 1
  }
}
```


POST /pedidos/{id}/pagamento

Processar Simulação de Pagamento (Mock)

CLIENTE (próprio pedido), ATENDENTE ou ADMIN. Recusa mantém reserva e permite nova tentativa. Aprovação avança para EM_PREPARO. Última tentativa em pagamento; histórico completo na auditoria. INDISPONIVEL retorna 503; TIMEOUT retorna 504. Falhas técnicas preservam pedido, pagamento anterior e reserva e são auditadas. TIMEOUT é simulado imediatamente, sem espera real.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Parâmetro id (path):

Request (exemplo):

```
{
  "statusSimulado": "APROVADO"
}
```

Respostas documentadas: 200, 400, 401, 403, 404, 409, 413, 422, 500, 502, 503, 504

Response 200 (exemplo):


```
{
  "sucesso": true,
  "mensagem": "Pagamento simulado aprovado.",
  "transacaoId": "MOCK-TX-exemplo",
  "statusPedido": "EM_PREPARO",
  "detalhes": {
    "id": 1,
    "pedidoId": 1,
    "valor": 91.0,
    "status": "APROVADO",
    "metodo": "SIMULACAO_GATEWAY",
    "transacaoId": "MOCK-TX-exemplo",
    "criadoEm": "2026-10-11T12:00:00.000Z"
  }
}
```


PATCH /pedidos/{id}/status

Atualizar Estado Operacional (Exclusivo Cozinha/Admin)

COZINHA ou ADMIN. EM_PREPARO → PRONTO → ENTREGUE; AGUARDANDO_PAGAMENTO → CANCELADO, com devolução de estoque. Este mock não implementa estorno de pedidos pagos.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Parâmetro id (path):

Request (exemplo):

```
{
  "status": "PRONTO"
}
```

Respostas documentadas: 200, 400, 401, 403, 404, 409, 413, 422, 500

Response 200 (exemplo):


```
{
  "id": 1,
  "usuarioId": 1,
  "unidadeId": 1,
  "canalPedido": "TOTEM",
  "status": "PRONTO",
  "total": 91.0,
  "criadoEm": "2026-10-11T12:00:00.000Z",
  "atualizadoEm": "2026-10-11T12:00:00.000Z",
  "itens": [
    {
      "id": 1,
      "pedidoId": 1,
      "produtoId": 1,
      "quantidade": 2,
      "precoUnitario": 38.5
    },
    {
      "id": 2,
      "pedidoId": 1,
      "produtoId": 2,
      "quantidade": 1,
      "precoUnitario": 14.0
    }
  ]
}
```


GET /unidades

Listar unidades

Autenticação: Pública.

Parâmetro page (query):

Parâmetro limit (query):

Respostas documentadas: 200, 409, 422, 500

Response 200 (exemplo):


```
{
  "dados": [
    {
      "id": 1,
      "nome": "Unidade Recife Olinda",
      "cidade": "Recife"
    }
  ],
  "paginacao": {
    "pagina": 1,
    "limite": 10,
    "total": 1,
    "totalPaginas": 1
  }
}
```


GET /health

Disponibilidade da API

Autenticação: Pública.

Respostas documentadas: 200

Response 200 (exemplo):


```
{
  "status": "ok"
}
```


GET /privacidade/aviso

Consultar aviso opcional da demonstração de fidelidade

Não ativa um programa real de pontos ou comunicação. Consulte a versão antes de aceitar.

Autenticação: Pública.

Respostas documentadas: 200

Response 200 (exemplo):


```
{
  "finalidade": "FIDELIDADE",
  "versaoAviso": "1.0",
  "demonstracao": true,
  "opcional": true,
  "titulo": "Preferência de fidelidade - demonstração acadêmica",
  "descricao": "Registrar sua decisão opcional sobre participação na demonstração de fidelidade.
Esta versão apenas registra a preferência: não acumula pontos, não concede benefícios e não envia
mensagens. Um programa real exigirá novo aviso e nova decisão.",
  "dados": [
    "Identificador do usuário autenticado",
    "Decisão de aceitar ou revogar",
    "Versão do aviso",
    "Data e hora da decisão"
  ],
  "revogacao": "Envie {\"aceito\":false} para PUT /api/privacidade/consentimentos/fidelidade.
Pedidos e login permanecem disponíveis.",
  "retencao": "O histórico é mantido para demonstrar as decisões durante o projeto. Não há expurgo
automático; consulte a política de privacidade da entrega para o procedimento de encerramento.",
  "compartilhamento": "Nenhum provedor externo recebe os dados de consentimento nesta demonstração."
}
```


GET /privacidade/consentimentos

Consultar meu estado e histórico de consentimento

Identidade vem do JWT. Histórico próprio paginado; estado atual é calculado independentemente da página. A consulta é auditada.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Parâmetro page (query):

Parâmetro limit (query):

Respostas documentadas: 200, 401, 409, 422, 500

Response 200 (exemplo):


```
{
  "dados": [
    {
      "id": 1,
      "usuarioId": 1,
      "finalidade": "FIDELIDADE",
      "versaoAviso": "1.0",
```

```
      "aceito": true,
      "demonstracao": true,
      "registradoEm": "2026-10-11T12:00:00.000Z"
    }
  ],
  "paginacao": {
    "pagina": 1,
    "limite": 10,
    "total": 1,
    "totalPaginas": 1
  },
  "atual": {
    "finalidade": "FIDELIDADE",
    "versaoAviso": "1.0",
    "aceito": true,
    "demonstracao": true,
    "ultimoRegistroId": 1
  }
}
```


PUT /privacidade/consentimentos/fidelidade

Registrar ou revogar minha decisão opcional

Somente o titular autenticado. Aceite exige a versão vigente. Revogação não exige versão atual e não bloqueia pedidos. PUT repetido não duplica evento. Histórico é de demonstração; flag legada não é autorização.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Request (exemplo):


```
{
  "aceito": true,
  "versaoAviso": "1.0"
}
```


Respostas documentadas: 200, 400, 401, 409, 413, 422, 500

Response 200 (exemplo):


```
{
  "id": 1,
  "usuarioId": 1,
  "finalidade": "FIDELIDADE",
  "versaoAviso": "1.0",
  "aceito": true,
  "demonstracao": true,
  "registradoEm": "2026-10-11T12:00:00.000Z"
}
```


GET /auditoria

Consultar auditoria (ADMIN)

Somente ADMIN. Filtro opcional por ação exata. Consulta gera seu próprio log depois de ler a página; esse novo log aparece em consultas subsequentes. Não retorna entidades de usuários.

Autenticação: JWT Bearer; permissões conforme descrição da operação.

Parâmetro page (query):

Parâmetro limit (query):

Parâmetro acao (query):

Respostas documentadas: 200, 401, 403, 409, 422, 500

Response 200 (exemplo):


```
{
  "dados": [
    {
      "id": 1,
      "acao": "CONSULTA_PEDIDOS",
      "usuarioId": 3,
      "timestamp": "2026-10-11T12:00:00.000Z",
      "detalhes": {
        "pagina": 1,
        "limite": 10,
        "quantidadeRetornada": 1,
        "escopo": "REDE"
      }
    }
  ],
  "paginacao": {
    "pagina": 1,
    "limite": 10,
    "total": 1,
    "totalPaginas": 1
  }
}
```


Envelope de erro


```
{
  "error": "ESTOQUE_INSUFICIENTE",
  "message": "Estoque insuficiente para o produto solicitado.",
  "details": [],
  "timestamp": "2026-10-11T12:00:00.000Z",
  "path": "/api/pedidos",
  "requestId": "exemplo-ilustrativo"
}
```


Exemplo ilustrativo de estrutura. HTTP 400/422 indica entrada inválida; 401/403, autenticação/autorização; 404, recurso ausente; 409, conflito; 502/503/504, falha de integração simulada.

8. Segurança, privacidade e auditoria

1. Escopo e estado da implementação

Esta entrega é uma demonstração acadêmica local, com contas fictícias. Implementa RF14/RF15 e documenta RNF04. Os controles descritos abaixo não equivalem a certificação de conformidade jurídica ou a uma política pronta para uma operação comercial real.

Implementado:

- Aviso público, com finalidade identificada e versão.

- Registro e revogação da decisão opcional pelo próprio titular autenticado.

- Histórico persistente de eventos, com data, versão e indicação de demonstração.

- Auditoria de consultas a pedidos, histórico de consentimentos e consulta administrativa de logs.

- Acesso aos logs somente por ADMIN, com paginação, filtro e projeção de metadados permitidos.

- Respostas da API com Cache-Control: no-store; erros inesperados não serializam exceções completas no console; o campo path dos erros não inclui query string.

Permanecem documentados como procedimentos/propostas: expurgo periódico, anonimização de dados reais, atendimento formal de direitos e política operacional para produção. Não foram criados jobs de exclusão, canais externos de atendimento, programa de pontos ou envio de marketing.

2. Inventário de dados e finalidade

Grupo	Dados persistidos	Finalidade	Acesso e minimização
Usuário	ID, nome, e-mail, hash de senha, perfil, criação e flag legada	Identificação, autenticação e autorização	Login retorna nome/e-mail/perfil do titular, nunca hash. Não são coletados CPF, endereço, telefone ou dados de cartão.
Pedido e itens	Usuário, unidade, canal, produtos, quantidades, preço, total, status e datas	Executar e demonstrar atendimento	Cliente consulta seus pedidos; operação consulta a rede. Não são incluídos nome/e-mail do cliente nas listagens de pedidos.
Pagamento mock	Pedido, valor, resultado, transação e data	Demonstrar aprovação/recusa e consistência de status	Nenhuma cobrança, cartão ou transmissão a provedor financeiro.
Consentimento	Titular por ID, finalidade, versão, decisão, data, demonstracao	Comprovar a preferência opcional dentro da demonstração	Titular só registra/consulta seu histórico. A identidade vem do JWT. Histórico não contém cópia de nome, e-mail ou token.
Auditoria	Ator por ID, ação, instante e metadados operacionais selecionados	Rastreabilidade e verificação de acesso/operações	ADMIN consulta; não há endpoint público de logs nem inclusão do objeto Usuario. Não são registrados corpo HTTP completo, IP, senha ou JWT nos logs de negócio.
JWT	ID, nome, perfil, emissão e expiração no token assinado	Autenticar chamadas	O token não é gravado na auditoria. Quem usa o cliente deve protegê-lo. Não há refresh/revogação central de tokens nesta versão.

O SQLite e seus backups não são cifrados por esta aplicação. A proteção local depende das permissões do sistema operacional; o banco e .env não devem ser publicados. Auditoria contém IDs vinculáveis, portanto não é tratada como dado anônimo.

3. Fundamentos para o trabalho acadêmico

A modelagem distingue tratamento necessário ao pedido e preferência opcional. Como hipóteses para uma futura operação, cadastro/pedido podem ser avaliados sob execução de contrato; a preferência opcional exige manifestação específica. Auditoria, conservação e eventuais obrigações precisam de fundamento e prazos definidos pelo responsável da operação. Não se pressupõe que um consentimento genérico autorize todas as finalidades.

A LGPD prevê demonstração da manifestação de vontade e possibilidade de revogação (art. 8º), além de regras para término/conservação e direitos do titular (arts. 15, 16 e 18). Esses dispositivos orientam a distinção entre revogar uma finalidade e apagar indiscriminadamente registros. Fonte oficial: Lei 13.709/2018 - texto compilado.

As hipóteses acima são decisões de modelagem para o relatório, sujeitas à avaliação do contexto real; os prazos acadêmicos da seção 6 não são apresentados como prazos legais.

4. Aviso e ciclo de consentimento

Finalidade: FIDELIDADE. Versão inicial: 1.0. O texto deixa claro que a aplicação apenas registra a preferência para demonstração: não ativa programa real, não acumula pontos e não envia mensagens. A implantação futura de outro tratamento exige novo aviso e nova decisão. O aviso publicado está em src/application/privacy/aviso.ts; versões publicadas devem ser preservadas no histórico do projeto.

Operação	Contrato
Ler aviso	GET /api/privacidade/aviso, público
Aceitar	PUT /api/privacidade/consentimentos/fidelidade, JWT, {"aceito":true,"versaoAviso":"1.0"}
Revogar	Mesma rota, {"aceito":false}; não é preciso aceitar ou informar a versão vigente
Consultar estado/histórico	GET /api/privacidade/consentimentos?page=1&limit=10, JWT, somente titular

Não há decisão positiva presumida: sem evento, atual.aceito=false. O booleano Usuario.consentimentoLGPD é legado, não é consultado para autorizar a finalidade e não é convertido automaticamente em consentimento. O seed novo usa false, não fabrica eventos e preserva os dados existentes. Os registros têm demonstracao=true, inclusive quando produzidos via API nas contas fictícias.

Eventos de aceite/revogação são adicionados ao histórico sem sobrescrever os anteriores. O estado atual usa o último evento, independentemente da página consultada. Aceite de versão antiga não ativa a versão atual. Revogação com versão antiga continua possível. Repetir a mesma decisão via PUT não duplica o último evento; nova decisão diferente cria um evento novo.

Evento e auditoria são gravados na mesma transação: falha no log reverte também o consentimento. Recusar/revogar não impede login, pedido ou pagamento. Campos extras como

usuarioId no corpo são rejeitados; filtros de identidade enviados na URL não ampliam o acesso ao histórico.

5. Auditoria de acesso

CONSULTA_PEDIDOS registra ator, data, página, limite, filtro de canal, quantidade retornada e escopo PROPRIOS/REDE. O log não copia os pedidos. Os eventos de criação, pagamento e status existentes são mantidos.

CONSULTA_CONSENTIMENTOS, CONSENTIMENTO_REGISTRADO e CONSENTIMENTO_REVOGADO registram a operação de privacidade. Os eventos de decisão referenciam ID do consentimento, finalidade, versão e caráter de demonstração.

GET /api/auditoria?acao=CONSENTIMENTO_REVOGADO&page=1&limit=10 exige ADMIN. O filtro é por ação exata, em letras maiúsculas/sublinhado, até 64 caracteres. Cada leitura gera CONSULTA_AUDITORIA após selecionar a página; esse evento aparece em leituras seguintes. Assim, totais podem aumentar entre requisições, como em qualquer histórico ativo.

A resposta projeta somente chaves operacionais previstas. Campos arbitrários de registros legados e JSON inválido não são devolvidos como texto bruto. Não existe API de edição/exclusão dos logs; isso não torna o banco local imutável contra quem tem acesso direto ao arquivo.

6. Retenção e anonimização: política proposta para a demonstração

Grupo	Regra proposta para o ambiente acadêmico	Execução atual
Contas, pedidos, pagamentos, decisões e auditoria fictícios	Manter até a avaliação e eventual revisão; revisar necessidade até 30 dias após o encerramento da avaliação	Revisão manual; nenhum job de expurgo implementado
Bases de testes isoladas	Excluir o arquivo ao finalizar a suíte	Implementado pelo teste; arquivos remanescentes após interrupção devem ser revisados manualmente
Cópias de segurança da demonstração	Mesma revisão e destino da base de origem; restringir acesso e não publicar	Procedimento manual
Evidências acadêmicas	Preferir dados fictícios e exemplos sem tokens, senhas, segredos ou banco real	Revisão na preparação do PDF/repositório

Procedimento de encerramento: identificar a base correta e suas cópias, confirmar que contém apenas dados fictícios e que a avaliação terminou, separar evidências minimizadas necessárias, documentar a decisão de conservação ou descarte e executar o descarte autorizado. Nenhum dado existente foi apagado por esta etapa.

Se forem usados dados reais futuramente, estabelecer matriz de retenção por finalidade e fundamento antes da coleta. Uma proposta de anonimização deve remover identificadores diretos, revisar campos livres e relações que permitam reidentificação e validar o resultado. Trocar nome/e-mail mantendo vínculos identificáveis é pseudonimização, não garantia de anonimização. Não há mecanismo automático implementado aqui.

7. Procedimento proposto para direitos do titular

O titular pode consultar e revogar a preferência pelo próprio token. Para acesso ampliado, correção ou eliminação, o futuro responsável deverá definir canal, verificar identidade sem coleta excessiva, localizar dados em tabelas/logs/backups, avaliar conservação necessária, executar a medida aplicável e registrar a resposta. O projeto não inventa endereço de atendimento nem contato de encarregado.

8. Migração e verificação

20261011133230_consentimentos cria somente tabela e índice de consentimentos, com FK para usuários. Não altera ou apaga contas/pedidos existentes. Em outra instalação, executar npm run prisma:deploy e npm run prisma:generate (ou npm run setup).

As verificações cobrem aviso, flag legada, idempotência, revogação, versões, isolamento por titular, bloqueio de atribuição a outro usuário, permissões ADMIN, auditoria de leituras, filtragem de dados legados, rollback de evento quando auditoria falha e sanitização de erros inesperados. O resultado final está registrado no plano de testes da etapa 4.

9. Situação dos requisitos

- RF14: auditoria de leituras e consulta administrativa implementadas.

- RF15: finalidade, versão, registro/revogação e histórico implementados como demonstração opcional.

- RNF04: inventário, hipóteses de tratamento, minimização, retenção/anonimização e procedimento de direitos documentados; limitações operacionais explícitas.

- Cadastro público, fidelização real e promoções permanecem propostas da etapa 1.

9. Plano de testes

Ambiente e execução

Requisitos: Node.js 20+, dependências instaladas, migrations aplicadas e seed. API padrão: http://localhost:3000. Rodar npm run setup e npm run dev; importar postman/raizes_nordeste_clean_collection.json no Postman e executar **a coleção inteira, em ordem**, com uma iteração. As pastas preservam dependências de login, criação, pagamento, status e consentimento.

baseUrl é a única variável que precisa de ajuste para outra porta. token_cliente, token_cozinha, token_admin e pedidoId são capturados pelas requisições. Todas as contas são fictícias, com senha de demonstração 123456. Não exporte a coleção com valores reais de tokens preenchidos.

A coleção tem **32 cenários**, **17 com resposta 2xx** e **15 cenários de erro**, em 9 pastas. Recusa de pagamento é resultado de negócio respondido com HTTP 200 e possui asserção explícita sucesso=false.

Para validação automatizada: npm run build e npm test. A suíte HTTP usa banco SQLite temporário e executa também todas as requisições e asserções desta coleção. A execução automatizada reproduz os recursos pm usados aqui; não é uma alegação de execução pela interface do aplicativo Postman. O uso manual no Postman é reproduzível com o arquivo entregue.

A coleção manual usa a base da API em execução e consome uma unidade do produto 1 por execução bem-sucedida. O seed preserva saldo e pedidos; ele não repõe estoque. Use uma base separada nova para avaliações repetidas, sem apagar a base existente. IDs 99999 usados nos casos 404 pressupõem o conjunto do seed, sem recursos com esses IDs.

Matriz de cenários e evidências

ID	Cenário	Endpoint e método	Pré-condição	Entrada	Saída esperada	Evidência na coleção
T01	Login cliente	POST /api/auth/login	API iniciada; migrations e seed aplicados	{"email":"maria@cliente.com","senha":"123456"}	200; accessToken presente; CLIENTE	01 Auth/T01 - Login cliente
T02	Login cozinha	POST /api/auth/login	API iniciada; migrations e seed aplicados	{"email":"cozinha@raizes.com","senha":"123456"}	200; accessToken presente; COZINHA	01 Auth/T02 - Login cozinha
T03	Senha incorreta	POST /api/auth/login	API iniciada; migrations e seed aplicados	{"email":"maria@cliente.com","senha":"errada"}	401; CREDENCIAIS_INVALIDAS	01 Auth/T03 - Senha incorreta
T04	Cardápio	GET /api/produtos	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	200; dados e paginacao; estoque por unidade	02 Cardápio/T04 - Cardápio
T05	Criar pedido TOTEM	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"TOTEM","itens":[{"produtoId":1,"quantidade":1}]}	201; id capturado; AGUARDANDO_PAGAMENTO; total 38,50	03 Pedidos/T05 - Criar pedido TOTEM
T06	Filtrar pedidos	GET /api/pedidos?canalPedido=TOTEM&page=1&limit=5	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	Sem body; parâmetros na rota	200; canal TOTEM nos resultados	03 Pedidos/T06 - Filtrar pedidos
T07	Canal inválido	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"INVALIDO","itens":[{"produtoId":1,"quantidade":1}]}	400; CANAL_INVALIDO	03 Pedidos/T07 - Canal inválido
T08	Estoque insuficiente	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"TOTEM","itens":[{"produtoId":1,"quantidade":99999}]}	409; ESTOQUE_INSUFICIENTE	03 Pedidos/T08 - Estoque insuficiente
T09	Quantidade inválida	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"TOTEM","itens":[{"produtoId":1,"quantidade":0}]}	422; PARAMETRO_INVALIDO	03 Pedidos/T09 - Quantidade inválida

ID	Cenário	Endpoint e método	Pré-condição	Entrada	Saída esperada	Evidência na coleção
T10	Pagamento recusado	POST /api/pedidos/{{pedidoId}}/pagamento	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pedido aguardando pagamento	{"statusSimulado":"RECUSADO"}	200; sucesso=false; AGUARDANDO_PAGAMENTO	04 Pagamento mock/T10 - Pagamento recusado
T11	Gateway indisponível	POST /api/pedidos/{{pedidoId}}/pagamento	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pedido aguardando pagamento	{"statusSimulado":"INDISPONIVEL"}	503; GATEWAY_INDISPONIVEL	04 Pagamento mock/T11 - Gateway indisponível
T12	Timeout gateway	POST /api/pedidos/{{pedidoId}}/pagamento	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pedido aguardando pagamento	{"statusSimulado":"TIMEOUT"}	504; GATEWAY_TIMEOUT	04 Pagamento mock/T12 - Timeout gateway
T13	Nova tentativa aprovada	POST /api/pedidos/{{pedidoId}}/pagamento	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pedido aguardando pagamento	{"statusSimulado":"APROVADO"}	200; sucesso=true; EM_PREPARO	04 Pagamento mock/T13 - Nova tentativa aprovada
T14	Cliente sem permissão	PATCH /api/pedidos/{{pedidoId}}/status	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pagamento aprovado em T13	{"status":"PRONTO"}	403; ACESSO_NEGADO	05 Status e duplicidade/T14 - Cliente sem permissão

ID	Cenário	Endpoint e método	Pré-condição	Entrada	Saída esperada	Evidência na coleção
T15	Cozinha marca pronto	PATCH /api/pedidos/{{pedidoId}}/status	API iniciada; migrations e seed aplicados; {{token_cozinha}} obtido no login anterior; pedidoId capturado em T05; pagamento aprovado em T13	{"status":"PRONTO"}	200; status=PRONTO	05 Status e duplicidade/T15 - Cozinha marca pronto
T16	Entrega	PATCH /api/pedidos/{{pedidoId}}/status	API iniciada; migrations e seed aplicados; {{token_cozinha}} obtido no login anterior; pedidoId capturado em T05; pedido PRONTO após T15	{"status":"ENTREGUE"}	200; status=ENTREGUE	05 Status e duplicidade/T16 - Entrega
T17	Pagamento repetido	POST /api/pedidos/{{pedidoId}}/pagamento	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; pedidoId capturado em T05; pedido já ENTREGUE	{"statusSimulado":"APROVADO"}	409; ESTADO_INVALIDO	05 Status e duplicidade/T17 - Pagamento repetido
T18	Sem autenticação	GET /api/pedidos	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	401; NAO_AUTENTICADO	06 Acesso e paginação/T18 - Sem autenticação
T19	Produtos paginados	GET /api/produtos?unidadeId=1&page=2&limit=1	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	200; pagina=2; um produto	06 Acesso e paginação/T19 - Produtos paginados
T20	Unidades paginadas	GET /api/unidades?page=1&limit=1	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	200; uma unidade	06 Acesso e paginação/T20 - Unidades paginadas
T21	Aviso opcional	GET /api/privacidade/aviso	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	200; opcional=true; versão 1.0	07 Privacidade/T21 - Aviso opcional
T22	Aceitar demonstração	PUT /api/privacidade/consentimentos/fidelidade	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"aceito":true,"versaoAviso":"1.0"}	200; aceito=true	07 Privacidade/T22 - Aceitar demonstração

ID	Cenário	Endpoint e método	Pré-condição	Entrada	Saída esperada	Evidência na coleção
T23	Histórico próprio	GET /api/privacidade/consentimentos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior; aceite registrado em T22	Sem body; parâmetros na rota	200; atual.aceito=true	07 Privacidade/ T23 - Histórico próprio
T24	Revogar decisão	PUT /api/privacidade/consentimentos/fidelidade	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"aceito":false}	200; aceito=false	07 Privacidade/ T24 - Revogar decisão
T25	Histórico sem token	GET /api/privacidade/consentimentos	API iniciada; migrations e seed aplicados	Sem body; parâmetros na rota	401; NAO_AUTENTICADO	07 Privacidade/ T25 - Histórico sem token
T26	Cliente não acessa auditoria	GET /api/auditoria	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	Sem body; parâmetros na rota	403; ACESSO_NEGADO	08 Auditoria/T26 - Cliente não acessa auditoria
T27	Login administrador	POST /api/auth/login	API iniciada; migrations e seed aplicados	{"email":"admin@raizes.com","senha":"123456"}	200; accessToken presente; ADMIN	08 Auditoria/T27 - Login administrador
T28	Evidência de revogação auditada	GET /api/auditoria?acao=CONSENTIMENTO_REVOGADO&limit=1	API iniciada; migrations e seed aplicados; {{token_admin}} obtido no login anterior; revogação registrada em T24	Sem body; parâmetros na rota	200; evento CONSENTIMENTO_REVOGADO	08 Auditoria/T28 - Evidência de revogação auditada
T29	Canal obrigatório ausente	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"itens":[{"produtoId":1,"quantidade":1}]}	400; CANAL_INVALIDO	09 Validações adicionais/T29 - Canal obrigatório ausente
T30	Tipo de quantidade inválido	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"TOTEM","itens":[{"produtoId":1,"quantidade":"1"}]}	422; PARAMETRO_INVALIDO	09 Validações adicionais/T30 - Tipo de quantidade inválido
T31	Unidade inexistente	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":99999,"canalPedido":"TOTEM","itens":[{"produtoId":1,"quantidade":1}]}	404; UNIDADE_NAO_ENCONTRADA	09 Validações adicionais/T31 - Unidade inexistente

ID	Cenário	Endpoint e método	Pré-condição	Entrada	Saída esperada	Evidência na coleção
T32	Produto inexistente	POST /api/pedidos	API iniciada; migrations e seed aplicados; {{token_cliente}} obtido no login anterior	{"unidadeId":1,"canalPedido":"TOTEM","itens":[{"produtoId":99999,"quantidade":1}]}	404; PRODUTO_NAO_ENCONTRADO	09 Validações adicionais/T32 - Produto inexistente

Cobertura e rastreabilidade

Requisito	Cenários principais
RF01-RF02: autenticação e autorização	T01-T03, T14, T18, T25-T27
RF04-RF05/RF16: unidades, cardápio e paginação	T04, T19-T20
RF06-RF08: pedido, canal, estoque e validação	T05-T09, T29-T32
RF09: consulta por canal	T06; isolamento por cliente nos testes automatizados
RF10-RF11: gateway mock e falhas	T10-T13, T17
RF12: transições operacionais	T14-T16
RF13: cancelamento e devolução	Teste automatizado de cancelamento, devolução única e bloqueio de pagamento
RF14: auditoria	T28 comprova evento persistido; testes automatizados verificam ator, filtros e consultas
RF15: consentimento	T21-T25; testes automatizados verificam histórico, isolamento, revogação e rollback
RNF05/RNF08: erros e evidência executável	Todos os cenários de erro verificam status, envelope e código de erro

Testes automatizados complementares

tests/api.test.ts cobre itens repetidos, rollback de reserva, estoque de outra unidade, IDs parcialmente numéricos, paginação inválida, cliente acessando pedido alheio, cancelamento, seed idempotente, concorrência na última unidade e no pagamento, contrato enviado ao gateway, falha de correlação e preservação de pagamento anterior.

Também cobre flag legada ignorada, versão inválida de aviso, tentativa de definir identidade pelo body, histórico próprio, revogação sem bloquear pedido, permissões de auditoria, projeção segura de logs, rollback em falha de auditoria e erro interno sem exceção/query sensível. Os artefatos OpenAPI e Postman têm verificação de contratos e exemplos.

Critério de conclusão

Build e suíte devem terminar com código zero, sem testes falhando; coleção precisa manter os cenários acima e executar suas asserções; exemplos OpenAPI devem corresponder aos schemas. Não considerar prints ou contagens isoladas como substitutos da evidência executável. A execução final e suas contagens estão registradas em docs/04_RESULTADO_VALIDACAO.md.

10. Resultados e desempenho

Data: 11/10/2026. Escopo: RNF09, consultas de produtos e unidades com 20 clientes concorrentes durante 60 segundos.

Alteração da API

As listagens abriam transações interativas por meio de Store.transaction, executando página e contagem em chamadas separadas entre JavaScript e o engine. O ensaio curto anterior à alteração reproduziu 15 respostas 500 e 7 timeouts em 33 tentativas.

ListarUnidadesUseCase e ListarProdutosCardapioUseCase agora chamam diretamente os métodos de consulta da porta. PrismaStore executa página e contagem por $transaction([...]), em lote. Mantém a transação para consistência da paginação, sem manter uma transação interativa aberta entre callbacks da aplicação. Quando já existe um cliente transacional, as consultas usam a transação externa.

Não foram removidas as transações de criação de pedido, reserva/devolução de estoque, pagamento, consentimento ou auditoria. Não há cache de saldo nem alteração do contrato HTTP.

Correção do gerador de carga

Após a alteração da API, a execução com o cliente antigo apresentou p95 de 93,34 ms, mas registrou 8.763 URLError e uma resposta 500 em 31.874 tentativas. Esses resultados estão preservados em ensaio sem reuso. URLError sozinho não identifica a causa de sistema operacional; esgotamento de recursos por abertura de conexões é uma hipótese, não uma conclusão comprovada.

O gerador antigo abria uma conexão para cada requisição. O verificador agora mantém uma conexão HTTP por cliente, lê integralmente cada resposta, verifica conteúdo de sucesso e classifica erros HTTP e de transporte. Continua usando 20 clientes em ciclo fechado, mesmos endpoints, base isolada, janela de 60 segundos e timeout de 10 segundos. Em caso de falha, fecha a conexão antes da tentativa seguinte. Não faz repetição oculta nem exclui falhas da latência.

A metodologia de conexão mudou: a comparação de antes/depois envolve tanto o caminho de consulta quanto o cliente de carga, portanto não é uma estimativa isolada do ganho da API. O resultado atual serve para verificar a meta no cenário explicitado.

Evidências

Meta atingida: 48.486 consultas em janela de 60 segundos, p95 de 39,37 ms e zero erros (incluindo zero 5xx). Tempo total com conclusão das requisições: 60,26 s. O indicador rnf09_target_met é true.

- Resultado original: 151 tentativas, 138 erros e p95 de 10.015,89 ms.

- Resultado final: medição após as correções, com reutilização de conexões.

- Compilação TypeScript aprovada.

- Suíte com **37 testes aprovados**, sem falhas; inclui o novo cenário “20 consultas publicas concorrentes retornam paginas sem falhas”, estoque concorrente, pagamento concorrente e os 32 cenários Postman.

- OpenAPI: 12 operações e 81 exemplos validados; coleção: 32 cenários em 9 pastas.

- Reinício, restauração e integridade SQLite conferidos pelo verificador operacional.

Reproduzir com python scripts/verify_operations.py, após instalar as dependências. O relatório é substituído a cada execução completa; o processo sinaliza falha quando não atinge a meta. OPERATIONS_SECONDS permite diagnósticos curtos, salvos somente na pasta isolada de teste. Logs ficam em .test-output/operations-*/server.log.

O ensaio não representa carga de escrita, rede externa, base volumosa ou garantia de produção.

11. Instalação e recuperação

Instalação e configuração

Requisitos: Node.js 20 ou superior e npm; alternativamente pnpm com o lockfile existente. Execute na raiz do projeto:

```
npm install
npm run setup
npm run build
npm start
```

Para usar exatamente as versões do lockfile, substitua a instalação por pnpm install --frozen-lockfile e use pnpm run nos scripts. Não há lockfile npm; npm install resolve os intervalos do package.json. O primeiro setup cria .env com segredo aleatório, gera o cliente Prisma, aplica migrações existentes e executa seed idempotente. Não execute prisma migrate reset para preparar uma base com dados.

No Windows, powershell -ExecutionPolicy Bypass -File .\iniciar.ps1 automatiza preparação, build e início. -SomentePreparar não inicia o servidor. O fallback para runtime local é específico deste computador; em outro ambiente instale Node e npm. Para desenvolvimento use npm run dev.

Variável	Valor inicial / regra
PORT	3000; inteiro entre 1 e 65535
DATABASE_URL	file:./dev.db; arquivo relativo ao schema em prisma/
JWT_SECRET	Segredo aleatório externo ao código, mínimo de 32 caracteres; placeholder rejeitado
JWT_EXPIRES_IN	8h; inteiro positivo em segundos ou sufixo s, m, h, d
NODE_ENV	O seed é bloqueado quando igual a production

O setup preserva .env existente: corrigir um arquivo inválido é uma ação manual. Não publicar .env, bancos ou tokens. As contas fictícias e seus perfis estão no README; o seed não é um endpoint de cadastro.

Verificação inicial: abrir http://localhost:3000/api/health (esperado {"status":"ok"}), depois http://localhost:3000/api-docs. O health comprova resposta do processo; não executa consulta de prontidão no banco. O contrato JSON está em /api-docs.json. Encerrar com Ctrl+C. Alterar a porta exige atualizar a URL usada no navegador/Postman.

Backup, restauração e recuperação local

Procedimento para a configuração padrão, sem apagar a base original:

1. Encerrar a API com Ctrl+C e garantir que nenhum outro processo esteja escrevendo na base.

2. Criar uma pasta de backup e copiar prisma/dev.db para um arquivo com data. Caso existam arquivos WAL/journal ativos, não copiar apenas o arquivo principal; usar a API de backup do SQLite, como no verificador operacional.

3. Para verificar uma restauração, copiar o backup para **outro arquivo**, por exemplo prisma/restauracao-teste.db. Configurar temporariamente DATABASE_URL="file:./restauracao-teste.db" e outra porta, mantendo o mesmo

schema/migrações da cópia.

4. Iniciar a API, fazer login e confirmar os pedidos esperados, estados e totais. Executar PRAGMA integrity_check com uma ferramenta SQLite; resultado esperado ok.

5. Encerrar a instância de teste e restaurar a configuração original. Não sobrescrever a base ativa durante a validação.

O verificador operacional automatiza criação por migrações e seed em .test-output, criação de pedido, término do processo, reinício, backup via sqlite3.Connection.backup, restauração em outro arquivo e consulta HTTP. Ele não lê nem modifica prisma/dev.db. Mantém as cópias isoladas para inspeção; .test-output/ está ignorado pelo Git. Backup local no mesmo disco é uma demonstração técnica, não proteção contra perda do dispositivo.

12. Rastreabilidade

Requisitos funcionais

Os casos de uso citados estão em src/application/use-cases; seus nomes correspondem aos arquivos .ts.

RF	Situação	Código / artefato	Teste ou evidência
RF01 autenticar	Implementado	AutenticarUsuarioUseCase; TokenService	login rejeita senha incorreta e dados inválidos; login válido usado na preparação da suíte
RF02 autorizar	Implementado	authMiddleware; ProcessarPagamentoMockUseCase	rotas privadas exigem JWT válido e perfil permitido; cliente vê somente seus pedidos e não paga pedido alheio
RF03 cadastrar cliente	Proposto	Etapa 1, seção 7	Sem teste executável; contas originadas no seed
RF04 unidades	Implementado	ListarUnidadesUseCase	produtos e unidades paginam com totais, limites e páginas vazias
RF05 cardápio por unidade	Implementado	ListarProdutosCardapioUseCase	cardápio público retorna estoque separado por unidade
RF06 criar pedido	Implementado	CriarPedidoUseCase	cria pedido, calcula total e reserva somente na unidade escolhida; itens ausentes/nulos/inválidos
RF07 canal	Implementado	CriarPedidoUseCase; ListarPedidosUseCase	canal inválido e unidade inexistente; filtro por canal e paginação
RF08 estoque transacional	Implementado	CriarPedidoUseCase; PrismaStore	falha no segundo item reverte reserva; pedidos concorrentes não vendem última unidade duas vezes
RF09 consultar pedidos	Implementado	ListarPedidosUseCase	isolamento por cliente; filtro por canal e paginação
RF10 contrato de pagamento	Implementado	ProcessarPagamentoMockUseCase; PaymentGateway; MockPaymentGateway	caso de uso envia contrato ao gateway e usa seu retorno para aprovar
RF11 falhas e repetição	Implementado	ProcessarPagamentoMockUseCase	recusa permite nova tentativa; falhas técnicas preservam pedido; correlação inválida; pagamentos concorrentes
RF12 status	Implementado	AtualizarStatusPedidoUseCase	cozinha avança preparo para pronto e entregue sem regressão
RF13 cancelamento	Implementado	AtualizarStatusPedidoUseCase	cancelamento devolve estoque exatamente uma vez
RF14 auditoria	Implementado	ListarPedidosUseCase; ListarAuditoriaUseCase; AuditLogger	consultas registram ator/escopo/filtros; auditoria exige ADMIN e audita própria consulta
RF15 consentimento	Implementado	GerenciarConsentimentoUseCase; tabela Consentimento	aceitar/revogar preserva histórico; versão atual; rollback de auditoria; revogação não bloqueia pedidos

RF	Situação	Código / artefato	Teste ou evidência
RF16 paginação	Implementado	ListarUnidadesUseCase; ListarProdutosCardapioUseCase	totais, limites e páginas vazias
RF17 gestão de produtos/unidades	Proposto	Etapa 1, seção 7	Sem endpoints/testes de administração
RF18 ajustes administrativos de estoque	Proposto	Etapa 1, seção 7	Saída por pedido/devolução existem; ajuste administrativo não implementado
RF19 fidelidade	Proposto	Etapa 1, seção 7	Consentimento existe; pontos e resgate não implementados
RF20 promoções	Proposto	Etapa 1, seção 7	Sem campanhas/descontos executáveis

Requisitos não funcionais

RNF	Situação e evidência
RNF01 separação de camadas	Conferência de domínio/aplicação, portas e composição; arquitetura.
RNF02 integridade/migrações	Schema/migrações; testes de rollback, concorrência e seed idempotente; base operacional criada por migrações.
RNF03 credenciais/segredos	HashService, TokenService, authMiddleware e .gitignore; testes de acesso. Revisão da publicação efetiva permanece na etapa 8.
RNF04 ciclo de vida de dados	Etapa 3: inventário, minimização e política; retenção/anonimização sem expurgo automático.
RNF05 validação/erros	errorHandlerMiddleware e validações; testes de JSON malformado, rota desconhecida, IDs inválidos e exceção sem exposição de dados.
RNF06 OpenAPI	swagger.json; scripts/validate_docs.py: referências e 81 exemplos de 12 operações.
RNF07 instalação	README, setup, migrações e inicializador; build e preparação isolada verificados nesta etapa. Instalação de dependências em cópia limpa prevista na etapa 8.
RNF08 evidências reproduzíveis	Plano, 36 testes e coleção com 32 cenários; resultado registrado na etapa 4.
RNF09 consultas/concorrência	**Atendido no ensaio local após correção.** 20 clientes por 60 s: 48.486 consultas, p95 39,37 ms, zero erros/5xx. Paginação e testes de disputa preservados; 37 testes aprovados. Método e limites.
RNF10 recuperação	Reinício preserva pedido e restauração em outra base mantém ID/total/status consultáveis; integridade SQLite ok. Procedimento e relatório operacional. Sem promessa de disponibilidade contínua.
RNF11 gateway tolerante a falha simulada	MockPaymentGateway e ProcessarPagamentoMockUseCase; testes de indisponibilidade, timeout, correlação e nova tentativa.
RNF12 rastreabilidade	Este mapa, plano de testes e diagramas do estado implementado; propostas identificadas separadamente.

13. Conclusão e situação da entrega

O projeto fecha o fluxo de pedido, pagamento simulado e atualização de status, com persistência, estoque por unidade e controle de acesso. A separação em camadas permite manter regras independentes do transporte HTTP e da implementação concreta do gateway.

A validação funcional reuniu 37 testes aprovados e 32 cenários Postman. No ensaio corrigido de consultas públicas, 20 clientes durante 60 segundos produziram 48.486 consultas, sem erros, com p95 de 39,37 ms. A mudança do cliente de carga também contribui para a diferença metodológica; não se atribui todo o ganho exclusivamente à API.

O modelo permanece demonstrativo: não há pagamento real, cadastro público, pontos/resgate, campanhas, expurgo automático nem garantia de disponibilidade contínua. Essas extensões foram identificadas e especificadas conceitualmente.

Este PDF constitui a versão acadêmica local. O roteiro fornecido com o projeto foi adotado como referência oficial desta entrega por orientação do aluno. A instalação foi verificada em uma cópia limpa: dependências pelo lockfile congelado, geração do Prisma, migrações e seed em banco novo, compilação TypeScript e aprovação dos 37 testes. O repositório está em https://github.com/angelicafolgado/raizes-do-nordeste. O conteúdo final foi consolidado em um único commit de Angelica Dias Folgado; o push que substitui a história antiga ainda está pendente.

14. Referências e artefatos consultados

UNINTER. Roteiro de Atividade Prática de Projeto Multidisciplinar - Trilha Back-End. Arquivo fornecido pelo aluno e adotado como referência oficial desta entrega por orientação do aluno. Arquivo local: UNINTER_Roteiro_Back_End_Copia_Adaptada_Estudo.pdf. Consultado em 11 out. 2026.

RAÍZES DO NORDESTE. Código-fonte da API REST em TypeScript. Artefatos locais: src/, prisma/schema.prisma, prisma/migrations/ e tests/api.test.ts. Versão de trabalho consultada em 11 out. 2026.

RAÍZES DO NORDESTE. Documentação técnica do projeto. README.md e docs/01 a docs/06: requisitos, complementos, privacidade, plano de testes, diagramas, manual técnico e rastreabilidade. 2026.

RAÍZES DO NORDESTE. Contrato OpenAPI e coleção de testes. src/api/docs/swagger.json e postman/raizes_nordeste_clean_collection.json. 2026.

RAÍZES DO NORDESTE. Evidências operacionais. docs/06_RESULTADOS_OPERACIONAIS.json, relatório anterior e relatório do cliente sem reuso de conexões. 11 out. 2026.

Os diagramas foram gerados a partir do código e do schema local. As fontes editáveis DOT/Mermaid e as imagens PNG/SVG acompanham a pasta docs/diagramas/.